Algorithmen und Datenstrukturen | AlgDat

Zusammenfassung

INHALTSVERZEICHNIS

1. Suchbäume ... 3
   - 1.1. Binärer Suchbaum ... 3
     - 1.1.1. Traversierung ... 3
     - 1.1.2. Suche ... 3
     - 1.1.3. Einfügen ... 4
     - 1.1.4. Löschen ... 4
     - 1.1.5. Performance ... 4
     - 1.1.6. Arithmetische Progression ... 4
   - 1.2. AVL Baum ... 4
     - 1.2.1. Einfügen ... 5
     - 1.2.2. Rotieren ... 5
     - 1.2.3. Trinode Umstrukturierungs-Algorithmus ... 6
     - 1.2.4. Cut/Link Restrukturierungs-Algorithmus ... 6
     - 1.2.5. Löschen ... 7
     - 1.2.6. Performance ... 7
   - 1.3. Splay Baum ... 7
     - 1.3.1. Performance ... 8
2. Sorting ... 8
   - 2.1. Merge Sort ... 8
     - 2.1.1. Conquer ... 8
     - 2.1.2. Merge-Sort Baum ... 9
     - 2.1.3. Nicht-Rekursive implementierung ... 9
     - 2.1.4. Performance ... 10
   - 2.2. Quicksort ... 10
     - 2.2.1. Partitionierung ... 10
     - 2.2.2. Tree ... 10
     - 2.2.3. In-Place ... 11
     - 2.2.4. Performance ... 12
   - 2.3. Zusammenfassung ... 12
   - 2.4. Vergleichsbasierte Sortierung ... 12
   - 2.5. Bucket-Sort ... 13
     - 2.5.1. Eigenschaften ... 13
     - 2.5.2. Erweiterungen ... 14
     - 2.5.3. Performance ... 14
   - 2.6. Lexikographisch ... 14
     - 2.6.1. Ordnung ... 14
     - 2.6.2. Sortierung ... 14
   - 2.7. Radix-Sort ... 14
     - 2.7.1. Binäre Zahlen ... 14
3. Pattern matching ... 15
   - 3.1. Strings ... 15

3.2. Brute-Force Pattern Matching 15
3.2.1. Performance 15
3.3. Boyer-Moore 15
3.3.1. Heuristiken 15
3.3.2. Algorithmus 16

WARN:
This document is still under development and will be completed around 1.2.2027

1. SUCHBÄUME

Multimaps: Ungeordnet, `find(k)`, `findAll(k)`, `insert(k,o)`, `remove(e)`

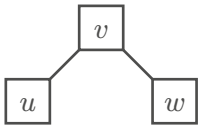
Geordnete Multimaps: Geordnet, `first()`, `last()`, `successors(k)`, `predecessors(k)`

Binäre Suche: Bei einer Multimap, realisiert als array-basierte Sequenz, sortiert nach Key, bei `find(k)`:
bei jedem Schritt wird die Anzahl der Kandidaten halbiert, terminiert nach $O(\log n)$ Schritten

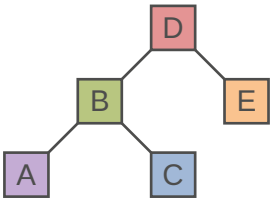
Suchtabelle: Multimap, welche mithilfe einer sortierten Sequenz implementiert wird.
- `find`: $O(\log n)$
- `insert`: $O(n)$
- `remove`: $O(n)$

1.1. BINÄRER SUCHBAUM

Ein binärer Baum, welcher Keys (oder Key- Value-Entries) in seinen internen Knoten speichert und folgende Bedingungen erfüllt:
Gegeben sind die drei Knoten u, v, und w. u ist im linken Teilbaum von v und w ist im rechten Teilbaum von v. So gilt: $\text{key}(u) \leq \text{key}(v) \leq \text{key}(w)$



1.1.1. Traversierung

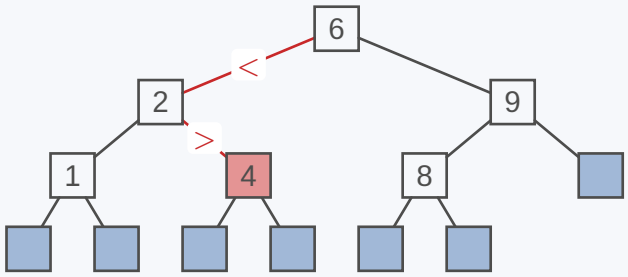


Preorder (W-L-R)	D B A C E
Postorder (L-R-W)	A C B E D
Inorder (L-W-R)	A B C D E
Breadth-First/ Level-Order	D B E A C

1.1.2. Suche

```
V search(K k, Node<V> n) {
  if (n.isExternal)
    return null;
  else if (k < n.key)
    return search(k, n.left);
  else if (k > n.key)
    return search(k, n.right);
  else
    return n.value;
}
```

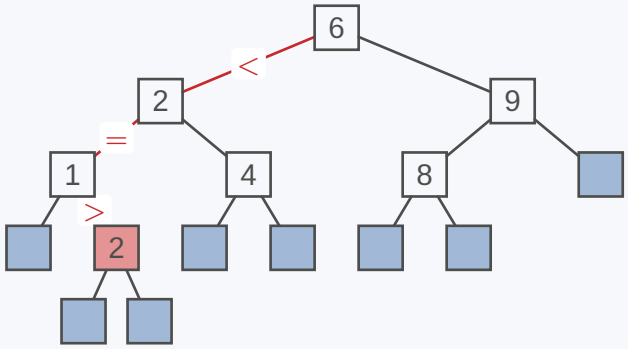
***Beispiel** 1:* `search(4)`



1.1.3. Einfügen

Annahme: der Baum ist eine Multimap und k ist schon im Baum vorhanden: Im jeweils linken Teilbaum von k wird weitergesucht bis man auf ein Blattknoten w stösst

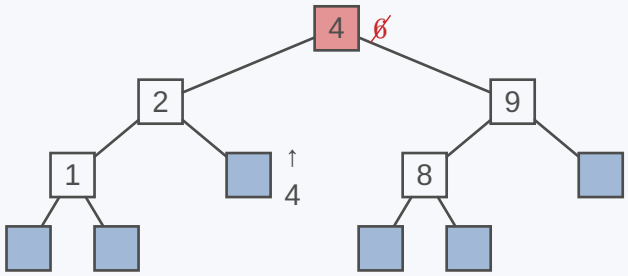
Beispiel 2: `insert(2)`



1.1.4. Löschen

– finde den internen Knoten w (4), welchem v (6) in der Inorder-Traversierung folgt
– kopiere $\text{key}(w)$ in den Knoten v
– lösche den Knoten w und sein rechtes Kind z (welches ein Blatt sein muss)

Beispiel 3: `delete(6)`



1.1.5. Performance

Betrachte eine Map mit n Entries, welcher mit einem binären Suchbaum implementiert ist mit der Höhe h
– nötiger Speicher ist $O(n)$
– Methoden `find`, `insert` und `remove` benötigen $O(h)$
– Die Höhe h ist $O(n)$ im schlechtesten Fall und $O(\log n)$ im besten Fall

1.1.6. Arithmetische Progression

Der Worst-Case tritt z.B. ein, wenn man aufsteigend sortierte Werte in einen binäreren Such-Baum einfügt. Laut der ***Gaussschen Summenformel*** $^{n(n+1)}/_{2}$ ist die Worst-Case Laufzeit somit $O(n^2)$.

TODO:
implementation?

1.2. AVL BAUM

Ein AVL Baum ist ein binärer Such-Baum, bei dem für jeden internen Knoten v von T gilt: die Höhe der Kinder von v unterscheiden sich höchstens um 1. AVL Bäume sind balanciert.

Die Höhe eines AVL Baumes T, der n Keys speichert, ist $O(\log(n))$. Die **minimale** Anzahl Knoten können wir als Funktion der Höhe h definieren:

$$n(1) = 1$$
$$n(2) = 2$$
$$n(h) = 1 + n(h-1) + n(h-2)$$

Dabei gilt

$$
\begin{aligned}
n(h-1) &> n(h-2) \\
n(h) &> 2n(h-2) \\
n(h) &> 2^{\frac{h}{2}-1} \\
h &< 2\log(n(h)) + 2 \\
\Rightarrow \text{Höhe} &\sim O(\log(n))
\end{aligned}
$$

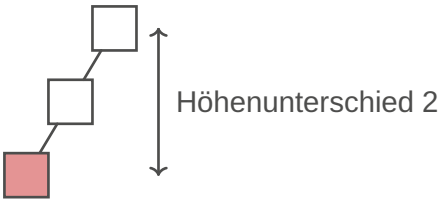
Balance

$$
\begin{aligned}
b(k) &= \text{Höhe(links)} - \text{Höhe(rechts)} \\
b(k) &\in \{-1, \emptyset, 1\} \\
-2 \leq b(k) &\leq 2 \qquad \text{Nach dem Einfügen eines neuen Knotens}
\end{aligned}
$$

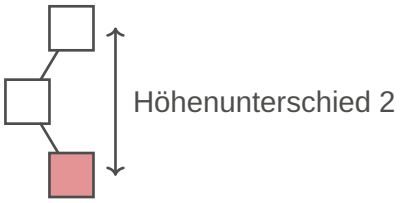
1.2.1. Einfügen

Wie beim binären Such-Baum. Verletzungen des AVL-Merkmals können beispielsweise auftreten beim:

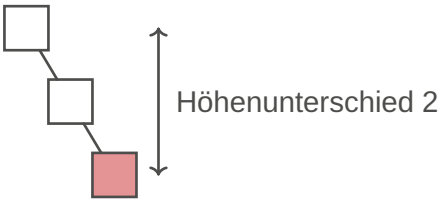
1) Einfügen eines Knotens in den linken Teilbaum des linken Sohnes



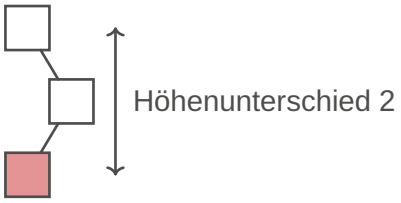
2) Einfügen eines Knotens in den rechten Teilbaum des linken Sohnes



3) Einfügen eines Knotens in den rechten Teilbaum des rechten Sohnes



4) Einfügen eines Knotens in den linken Teilbaum des rechten Sohnes



Nach dem Einfügen wandern wir vom neuen Knoten aus aufwärts, bis wir den ersten Knoten x finden, dessen Grosseltern z ein unbalancierter Knoten ist und balancieren den Baum aus.

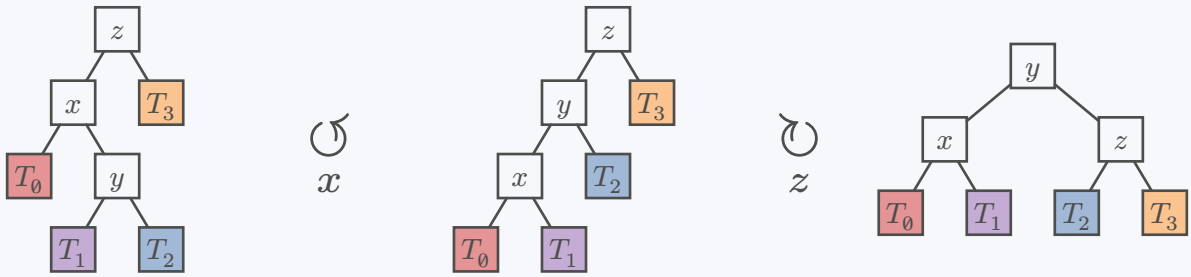
1.2.2. Rotieren

```
BinaryNode rotateWithLeftChild(BinaryNode k2) {
  BinaryNode k1 = k2.left;
  k2.left = k1.right;
  k1.right = k2;
  return k1;
}

BinaryNode doubleRotateWithLeftChild(BinaryNode k3) {
  k3.left = rotateWithRightChild(k3.left);
  return rotateWithLeftChild(k3);
}
```

Im Beispiel unten wird diese Rotation mit k3 = z, k2 = y und k1 = x durchgeführt.

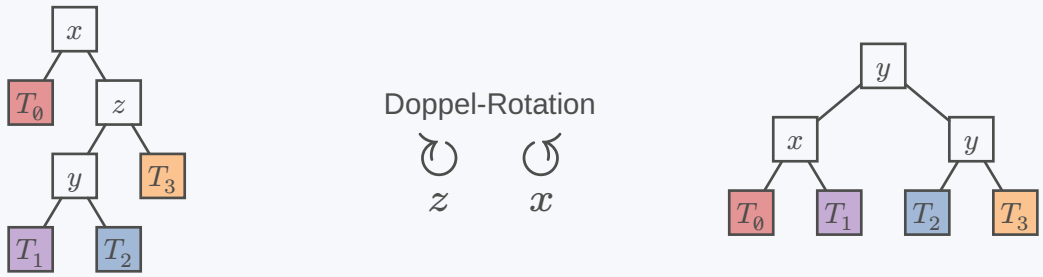
***Beispiel** 4:* `doubleRotateWithLeftChild(z);`



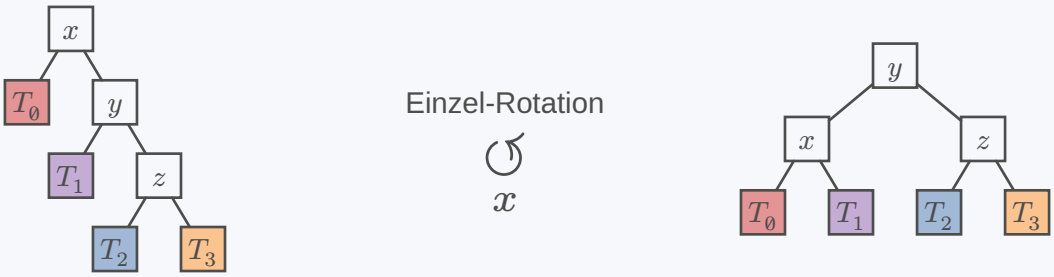
1.2.3. Trinode Umstrukturierungs-Algorithmus

Sei x, y, z die (Inorder) geordnete Liste der Knoten. Man führe die nötigen Rotationen durch, damit b zum obersten Knoten des Baumes wird.

***Beispiel** 5:* `doubleRotateWithRightChild(x);`



***Beispiel** 6:* `rotateWithRightChild(x);`

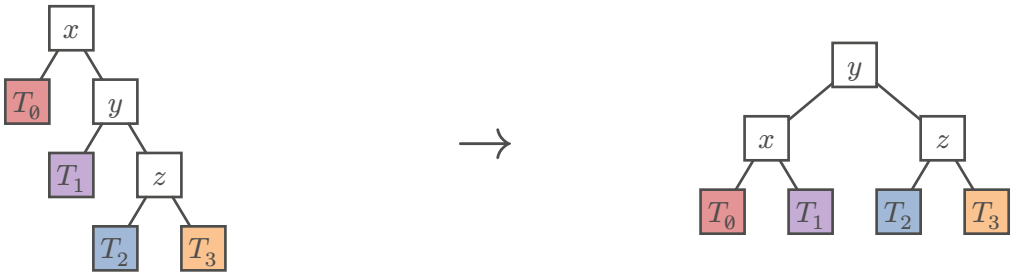


Weitere Fälle sind symmetrisch.

1.2.4. Cut/Link Restrukturierungs-Algorithmus

Sei x, y, z die (Inorder) geordnete Liste der Knoten, und sei (T_0, T_1, T_2, T_3) die Liste der vier Unterbäume von x, y, z.

1) Ersetze den Unterbaum mit Root x durch den Unterbaum mit Root y.
2) Setze x als linkes Kind von y und T_0, T_1 als den linken resp. rechten Unterbaum von x.
3) Setze z als rechtes Kind von y und T_2, T_3 als den linken resp. rechten Unterbaum von z.



Alternativ:

1) Kreiere ein Array mit 7 Elementen und befülle es In-Order mit den Knoten und den Unterbäumen:



2) Baue den Baum schrittweise wieder auf (beginnend bei y)

1.2.5. Löschen

Das Löschen eines Knotens w beginnt wie im binären Suchbaum. Sein Eltern-Knoten kann jetzt die Balance aus dem Gleichgewicht bringen, somit muss eine Umstrukturierung stattfinden. Die Umstrukturierung kann eine neue Unbalance hervorrufen bei Knoten höher im Baum. Somit muss die Balance weiter geprüft werden bis die Wurzel von T erreicht ist.

1.2.6. Performance

– Umstrukturierung ist $O(1)$
– `find` ist $O(\log n)$ (= Höhe des Baumes)
– `insert` ist $O(\log n)$ (`find` zu Beginn ist $O(\log n)$, eventuelle Restrukturierungen $O(1)$)
– `delete` ist $O(\log n)$ (`find` zu Beginn sowie eventuelle Restrukturierungen sind $O(\log n)$)

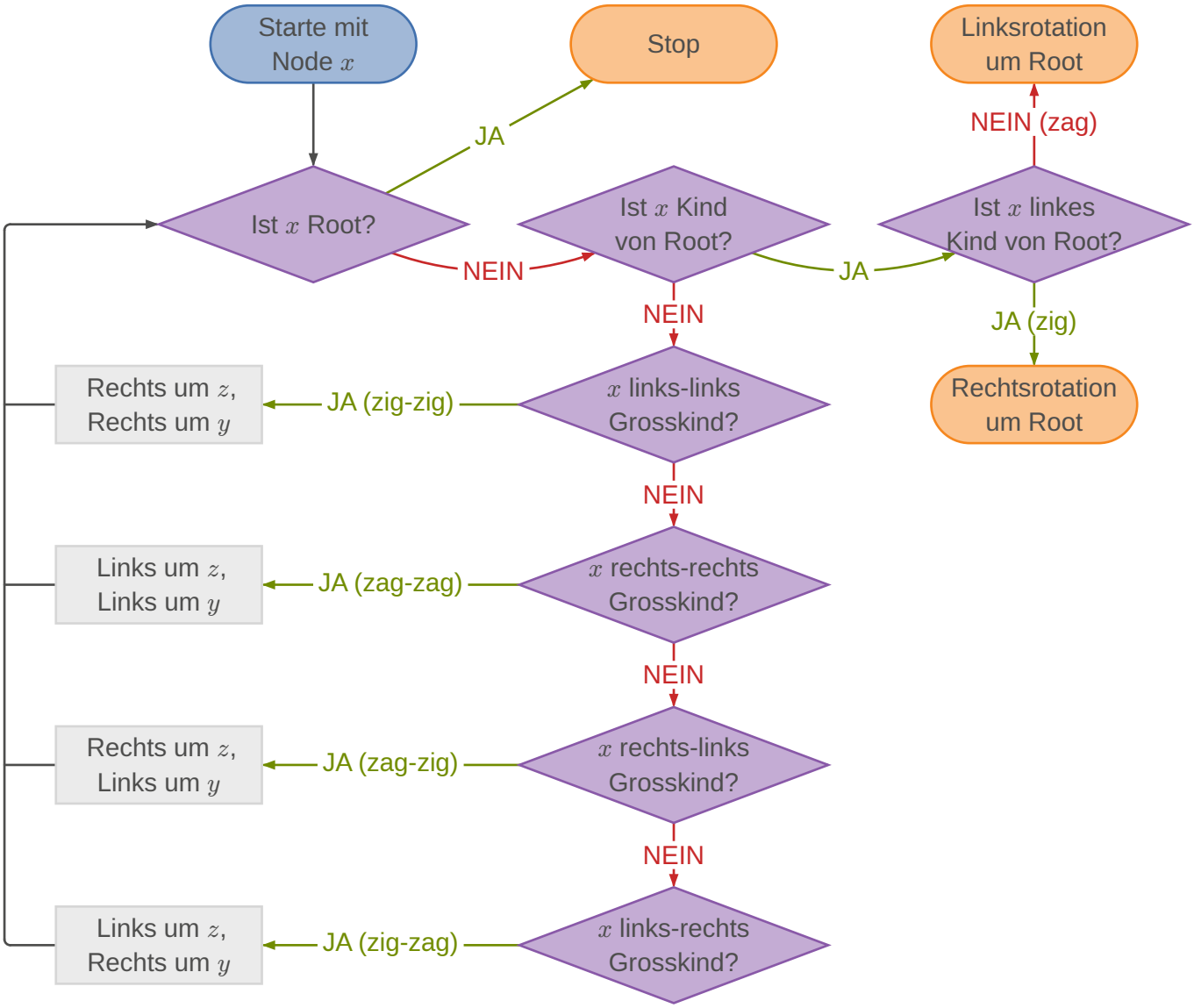
1.3. SPLAY BAUM

Ein Splay Baum ist ein binärer Such-Baum, bei welchem nach einem Zugriff auf einen Knoten dieser zur Root bewegt wird (jede Operation, auch Suche). Diese neue Operation heisst `splay`. Bewegt einen Knoten zum Root unter Benutzung von Rotationen.

Welcher Knoten wird «splayed» nach jeder Operation?

Methode	***Splay Knoten***
`find(k)`	Wenn Key gefunden, benutze diesen Knoten Wenn Key nicht gefunden, benutze den Eltern-Knoten des externen Knoten am Ende
`insert(k, v)`	Benutze den neuen Knoten bei welchem der Entry eingefügt/ersetzt wurde
`remove(k)`	Benutze den Eltern-Knoten des internen Knotens welcher gelöscht wurde

«x ist das links-rechts Grosskind»: x ist das linke Kind von seinem Eltern-Knoten, welcher selber ein rechtes Kind ist von seinem Eltern-Knoten. y ist x's Eltern-Knoten; z ist y's Eltern-Knoten



1.3.1. Performance

– `splay` ist $O(h)$
  – Durchschnittlich: $O(\log n)$, Für oft besuchte Knoten wesentlich schneller
  – Worst-Case: $O(h)$ Rotationen, jede mit $O(1)$

2. SORTING

2.1. MERGE SORT

Merge-sort ist ein Sortier-Algorithmus basierend auf dem Divide-and-Conquer Paradigma.

Divide: Input-Daten S in zwei getrennte Teilmengen S_1 und S_2 von je $^n/_2$ Elementen aufteilen

Recur: Die Teilprobleme mit S_1 and S_2 rekursiv lösen

Conquer: Mischen der Lösungen von S_1 und S_2 in die Lösung von S

2.1.1. Conquer

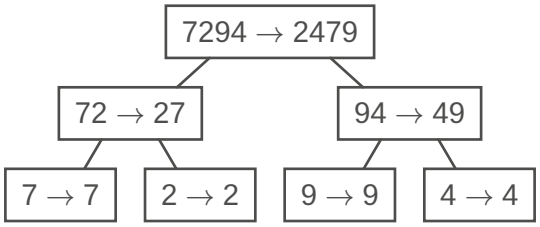
Mischen zweier sortierter Sequenzen von je $^n/_2$ Elemente mit double-linked Listen: $O(n)$ Laufzeit

```
void merge(int[] s1, int[] s2, int[] result) {
  int i = 0, j = 0, k = 0;
  while (i < s1.length && j < s2.length) {
    if (s1[i] ⩽ s2[j])
      result[k++] = s1[i++];
    else
      result[k++] = s2[j++];
  }
  while (i < s1.length)
    result[k++] = s1[i++];
  while (j < s2.length)
    result[k++] = s2[j++];
}
```

2.1.2. Merge-Sort Baum

Die Ausführung eines Merge-Sort kann als binärer Baum dargestellt werden:

- Jeder Knoten representiert einen rekursiven Aufruf des Merge-Sort und enthält
  - unsortierte Sequenz vor der Ausführung und der Aufteilung
  - sortierte Sequenz nach dem Ende der Ausführung
- die Wurzel entspricht dem initialen Aufruf
- die Blätter sind Aufrufe auf Teilsequenzen der Grösse 0 oder 1



2.1.3. Nicht-Rekursive implementierung

```
public static void mergeSort(Object[] orig, Comparator c) {
  Object[] in = new Object[orig.length];
  System.arraycopy(orig,0,in,0,in.length);    // copied temporary array
  Object[] out = new Object[in.length];       // output array
  Object[] temp;                              // temp array for swapping
  int n = in.length;
  for (int i=1; i < n; i*=2) {                // sorts all length-2*i runs
    for (int j=0; j < n; j+=2*i)              // merges two length-i pair
      merge(in,out,c,j,i); // merge from in to out two length-i runs at j
    temp = in; in = out; out = temp;          // swap arrays for next iter
  }
  System.arraycopy(in,0,orig,0,in.length);    // re-copy sorted "in" array
}
protected static void merge(Object[] in, Object[] out,
                            Comparator c, int start, int inc) {
  // merge in[start..start+inc-1] and in[start+inc..start+2*inc-1]
  int x = start;                              // index into run #1
  int end1 = Math.min(start+inc, in.length);  // boundary for run #1
```

```
  int end2 = Math.min(start+2*inc, in.length);// boundary for run #2
  int y = start+inc;                          // index into run #2
  int z = start;                              // index into the out array
  while (x < end1 && y < end2)
    if (c.compare(in[x],in[y]) ⩽ 0) out[z++] = in[x++];
    else out[z++] = in[y++];
  if (x < end1)                               // first run didn't finish
    System.arraycopy(in, x, out, z, end1 - x);
  else if (y < end2)                          // second run didn't finish
    System.arraycopy(in, y, out, z, end2 - y);
}
```

2.1.4. Performance

– Die Höhe h des Merge-Sort Baumes ist $O(\log n)$
  – bei jedem rekursiven Aufruf: Aufteilung in zwei Hälften
– Der Gesamt-Aufwand aller Knoten einer Tiefe i ist $O(n)$:
  – Aufteilung und Mischen von 2^i Sequenzen der Grösse $n/2^i$
  – $2^i + 1$ rekursive Aufrufe
– Somit: totale Laufzeit des Merge-Sort ist $O(n \log n)$

2.2. QUICKSORT

Quick-sort ist ein Sortier-Algorithmus basierend auf dem Divide-and-Conquer Paradigma:

Divide: Auswahl eines zufälligen Elementes x (Pivot) und Aufteilung von S in
  – L Elemente kleiner als x
  – E Elemente gleich x
  – G Elemente grösser als x

Recur: sortiere L und G

Conquer: vereine L, E and G

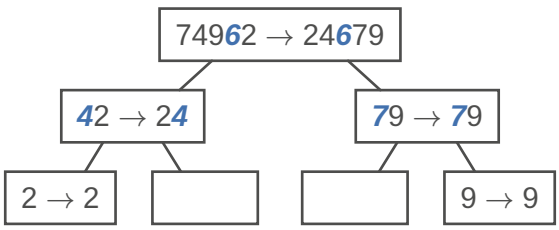
2.2.1. Partitionierung

Entferne jeweils ein Element y von S und füge es ein in L, E oder G, abhängig vom Resultat des Vergleichens mit dem Pivot x.

2.2.2. Tree

Die Ausführung eines Quick-Sort kann als binärer Baum dargestellt werden. Jeder Knoten repräsentiert einen rekursiven Aufruf des Quick-Sort und enthält:
– unsortierte Sequenz vor der Ausführung und sein Pivot
– sortierte Sequenz und sein Pivot nach dem Ende der Ausführung

Die Wurzel entspricht dem initialen Aufruf und die Blätter sind Aufrufe auf Teilsequenzen der Grösse 0 or 1.

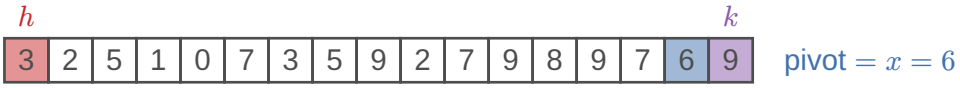


2.2.3. In-Place

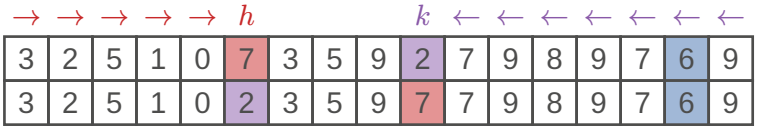
Quick-Sort kann «In-Place» implementiert werden. Im Partitionierungs-Schritt werden die Elemente der Input-Sequenz derart umgeordnet, dass:
– die Elemente $\leq$ Pivot einen Index kleiner als h haben
– die Elemente $\geq$ Pivot einen Index grösser als k haben

Die rekursiven Calls betrachten:
– Elemente mit Index kleiner als h
– Elemente mit Index grösser als k

Ablauf der Partitionierung:



Wiederholung bis h und k sich kreuzen:
– h nach rechts bis zu einem Element $> x$ oder gekreuzt
– k nach links bis zu einem Element $< x$ oder gekreuzt
– Wenn h und k noch nicht gekreuzt: Elemente mit Indizes h und k vertauschen



Pivot vertauschen mit Element bei Index:
– Wenn Pivot in L-Sequenz (links): k
– Wenn Pivot in G-Sequenz (rechts): h




```
void quickSort(Object[] S, Comparator c) {
  if (S.length < 2) return;            // array is already sorted
  quickSortStep(S, c, 0, S.length-1); // recursive sort method
}
void quickSortStep(Object[] S, Comparator c, int l, int r) {
  if (l ≥ r) return;                   // the indices have crossed
  Object temp;                         // temp object used for swapping
  Object pivot = S[r];
  int h = l;                           // will scan rightward
  int k = r - 1;                       // will scan leftward
  while (h ≤ k) {
    while (h ≤ k && c.compare(S[h], pivot) ≤ 0) h++;
    while (k ≥ h && c.compare(S[k], pivot ≥ 0) k--;
    if (h < k) {                       // both elements were found
      temp = S[k];
      S[k] = S[h];                     // swap these elements
      S[h] = temp;
    }
  }
  temp = S[r];                         // swap pivot with the element at h
  S[r] = S[h];
  S[h] = temp;                         // the pivot is now at h, so recurse
```

```
  quickSortStep(S, c, l, h - 1);
  quickSortStep(S, c, k + 1, r);
}
```

2.2.4. Performance

Aufteilung: Jedes Einfügen und Entfernen jeweils am Anfang oder am Ende der Sequenz: $O(1)$. Die Aufteilung des Quick-Sort benötigt somit: $O(n)$

Worst-Case: Worst-Case tritt auf, wenn das Pivot das Minimum- oder Maximum-Element ist. Eines von L oder G hat dann die Länge $n-1$, das andere $\emptyset$. Die Laufzeit ist proportional zu der Summe

$$n + (n+1) + ... + 2 + 1 = \sum_{i=\emptyset}^{n} i = \frac{n^2+n}{2} \sim O(n^2)$$

Erwartet: Ein rekursiver Aufruf von Quick-Sort auf einer Sequenz der Länge s ist ein Good call, wenn die Länge von L und G beide kleiner als $^3/_4 s$ sind, oder ein Bad call, wenn entweder L oder G länger als $^3/_4 s$ ist. Ein Aufruf ist «good» mit einer Wahrscheinlichkeit von $^1/_2$.
Mit der erwarteten Höhe des Quick-Sort Baumes von $O(\log n)$ und dem Gesamtaufwand für alle Knoten von $O(n)$ ergibt sich die totale Laufzeit von $O(n \log n)$

TODO:
W6 S14 (diagram)

2.3. ZUSAMMENFASSUNG

Algorithmus	*Zeitverhalten*	*Bemerkungen*
selection-sort	$O(n^2)$	– langsam – in-place – für kleine Data Sets ($< 1K$)
insertion-sort	$O(n^2)$	– langsam – in-place – für kleine Data Sets ($< 1K$)
quick-sort	$O(n \log n)$	– schnellster – in-place – für grosse Data Sets ($1K - 1M$)
heap-sort	$O(n \log n)$	– schnell – in-place – für grosse Data Sets ($1K - 1M$)
merge-sort	$O(n \log n)$	– schnell – sequentieller Datenzugriff – für riesige Data Sets ($> 1M$)

2.4. VERGLEICHSBASIERTE SORTIERUNG

Viele Sortier-Algoritmen sind vergleichsbasiert. Beispiele: Bubble-Sort, Selection-Sort, Insertion-Sort, Heap-Sort, Merge-Sort, Quick-Sort, … Es soll eine untere Grenze (Lower Bound) der Laufzeit hergeleitet werden für alle Algorithmen, welche Vergleiche benutzen um n Elemente $x_1, x_2, ..., x_n$ zu sortieren.

TODO:
W7 SortingLowerBound S3,4,5

Wir zählen die Anzahl der Vergleiche! Jeder mögliche Durchgang des Algorithmus korrespondiert mit einem Wurzel-zu-Blatt Pfad in einem Entscheidungs-Baum

Jeder Vergleichs-basierte Sortier-Algorithmus hat eine minimale Laufzeit von: $\log(n!)$. Jeder solche Algorithmus hat somit eine Laufzeit von mindestens:

$$\log(n!) \geq \log\left(\left(\frac{n}{2}\right)^{\frac{n}{2}}\right) = \frac{n}{2}\log\left(\frac{n}{2}\right)$$

TODO:

$$\log n! > n \log n$$
$$\log n! \sim n \log n - n$$

Oder gemäss Stirling-Annäherung:

$$n! \geq \sqrt{2\pi n}\left(\frac{n}{e}\right)^n \sim O(n \log n)$$

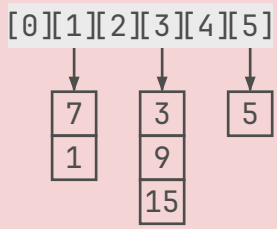
$\Rightarrow$ vergleichs-basierte Sortierung hat eine untere Grenze der Laufzeit von $\Omega(n \log(n))$.

TODO:
O = grösser gleich, Ω = kleiner gleich

2.5. BUCKET-SORT

Sei S eine Sequenz von n (k, v) Entries mit Key im Bereich von $[0, N-1]$. Bucket-Sort benutzt die Keys als Index in einem Hilfs-Array B von Sequenzen.
– Phase 1: Sequenz S leeren durch Verschieben jedes Eintrags (k, v) in sein Bucket B_k
– Phase 2: Für $i = 0, ..., N-1$, verschiebe die Entries des Buckets B_i an das Ende der Sequenz S

TODO:



TODO:
W7 BucketSort S2..6, implementation

2.5.1. Eigenschaften

– Keys werden als Indices in einem Array benutzt und können somit nicht beliebige Objekte sein.
– Es ist kein externer Comparator nötig.
– Stabile Sort Eigenschaft: Die relative Ordnung von zwei Entries mit dem selben Key werden durch den Algorithmus nicht verändert.

2.5.2. Erweiterungen

Integer Keys im Bereich $[a, b]$***:*** Stecke Entry (k, v) in Bucket B_{k-a}.

String-Keys: Sei D ein Set von möglichen Strings, wobei D eine konstante Grösse hat (z.B. die Namen der 50 U.S. Staaten)
- Sortiere D und berechne den Index $r(k)$ jedes Strings k von D in der sortierten Sequenz
- Füge Entry (k, v) in Bucket $B_{r(k)}$ ein

2.5.3. Performance

- Phase 1 benötigt $O(n)$
- Phase 2 benötigt $O(N + n)$

Insgesamt braucht es also $O(N + n)$

2.6. LEXIKOGRAPHISCH

2.6.1. Ordnung

Die lexikographische Ordnung von zwei d-Tupels ist rekursiv definiert als:

$$(x_1, x_2, ..., x_d) < (y_1, y_2, ..., y_d)$$
$$x_1 < y_1 \vee (x_1 = y_1 \wedge (x_2, ..., x_d) < (y_2, ..., y_d))$$

2.6.2. Sortierung

- Sei C_i der Comparator welcher zwei Tupel nach ihrer i-ten Dimension vergleicht.
- Sei $\text{stableSort}(S, C)$ ein stabiler Sortier-Algorithmus welcher den Comparator C benutzt.
- Lexikographische-Sortierung sortiert eine Sequenz von d-Tupeln in lexikographischer Ordnung, indem d-mal stableSort, je einmal pro Dimension, durchgeführt wird.
- Lexikographische-Sortierung läuft in $O(d * T(n))$ Zeit, wobei $T(n)$ die Laufzeit von stableSort ist.

TODO:
W7 RadixSort S6 algo + ex

2.7. RADIX-SORT

Radix-Sort ist eine Spezialisierung des lexikographischen-Sort, welcher Bucket-Sort als stabilen Sortier-Algorithmus für jede Dimension benutzt.

Radix-Sort ist anwendbar für Tupel mit Integer-Keys im Bereich $[0, N - 1]$ in jeder Dimension i und läuft in $O(d(n + N))$ Zeit.

TODO:
W7 RadixSort S7 algo + ex

2.7.1. Binäre Zahlen

Gebeben sei eine Sequenz von n b-Bit Integers

$$x = x_{b-1}...x_1x_0$$

Jedes Element wird als b-Tupel von Integern im Bereich $\{0, 1\}$ dargestellt. Darauf wendet man den Radix-Sort mit $N = 2$ an.

Diese Anwendung des Radix-Sort-Algorithmus läuft in $O(b \cdot n)$ Zeit.

TODO:
Beispiel: eine Sequenz von 32-Bit Integers kann in linearer Zeit sortiert werden.

W7 RadixSort S8,9 algo + ex

3. PATTERN MATCHING

3.1. STRINGS

Ein Alphabet Σ ist ein Set von möglichen Zeichen für eine Familie von Strings, z.B. ASCII.

Sei T ein String der Länge m.

Substring: Ein Substring $\{T_i, ..., T_j\}$ von T ist die Subsequenz von T, bestehend aus den Zeichen mit Index zwischen und inklusiv i und j.

Präfix: Ein Präfix von T ist ein Substring vom Typ $\{T_0, ..., T_j\}$

Suffix: Ein Suffix von T ist ein Substring vom Typ $\{T_i, ..., T_{m-1}\}$

Gegeben: Strings T (Text) und P (Pattern, Muster).

Das Pattern-Matching Problem besteht darin, einen Substring in T zu finden, der mit P übereinstimmt.

Anwendungen: Text Editoren, Suchmaschinen, Biologische Forschung

3.2. BRUTE-FORCE PATTERN MATCHING

Der Brute-Force Pattern Matching Algorithmus vergleicht das Pattern P mit dem Text T für jede mögliche Position von P relativ zu T, bis entweder
– eine Übereinstimmung gefunden wurde, oder
– alle möglichen Platzierungen des Patterns ausprobiert wurden.

TODO:
W7 PatternMatching S5 algo + ex

3.2.1. Performance

Brute-Force Pattern Matching benötigt $O(nm)$ Zeit.

3.3. BOYER-MOORE

3.3.1. Heuristiken

Der Boyer-Moore Pattern Matching Algorithmus basiert auf zwei Heuristiken:

Looking-Glass: Vergleiche P mit einer Subsequenz von T. Starte dabei am Ende des Patterns.

Character-Jump: Falls bei $T_i = c$ keine Übereinstimmung:
- falls P das Zeichen c enthält, verschiebe P bis das letzte Auftreten von c in P mit T_i übereinstimmt.
- sonst, verschiebe P bis P_0 mit T_{i+1} übereinstimmt.

TODO:
W7 PatternMatching S7 ex

3.3.2. Algorithmus

TODO:
W7 PatternMatching S8..